
Online Social Bookstore Application

Sprint Report

< . >

<Κωσταντίνος Μήτσης 4734 , Ορέστης Στέφας 4916, Γεώργιος Παρουτιάδης 4775>

VERSIONS HISTORY

Date	Version	Description	Author
17/5	1	Writing information about the project for the first time, after achieving what we could considering the time we spend on the code functionality.	All of us

1 Introduction

This project is intended to represent a simulation of a bookstore site where users can offer books for free and so that others can request, if they can find through search engine the book they want, or get a recommendation based on their favorite authors and/or book categories. Users can check any requests they have for the book they are offering. It is in their own free will to choose to whom they going to accept to get their book. Necessary information of the requesters is provided to the user. Every time a request is accepted, the users get a notification if accepted or declined, accordingly. Of course, users are able to delete any offers or requests they regretted, but accept notification cannot be undone. That is how the bookstore works.

1.1 Purpose

Educating reasons. Getting an idea of Springboot and how users interaction affects the logic of the back-end program. In the end, it is *what you want* to achieve in front, that you have to *work on the background* for it.

1.2 Document Structure

The rest of this document is structured as follows. Section 2 describes out Scrum team and specifies this Sprint's backlog. Section 3 specifies the main design concepts for this release of the project.

2 Scrum team and Sprint Backlog

2.1 Scrum team

Product Owner	Giorgos Paroytiadis
Scrum Master	Orestis Stefas
Development Team	Konstantinos Mitsis

2.2 Sprints

<List below the sprints that you performed and the user stories that have been realized in each Sprint>

Sprint No	Begin Date	End Date	Number of weeks	User stories
1			2	No user stories. We got stacked setting up the database and the project files due to lack of experience error we made and couldn't find solution.
2			1	No user stories. Fighting against the code needed for the config folder in order to begin interacting with the initial user stories.
3			1	User stories 1,2,3,4 completed.
4			4-5	Remaining user stories completed. Some of them are not perfected.

3 Use Cases

<Specify the concrete Use Cases that describe the interaction of the user with the applications, as derived from the abstract user stories. Give a **UML Use Case diagram** and the **detailed use case descriptions**.>

3.1 <Register>

Use case ID	registerUC
Actors	a user of the application
Pre conditions	No precondition required. Just the user must have internet connection. A name and a password to put on.
Main flow of events	1. The use case starts when the user connects to the localhost page on internet and pressed the register button. 2. User fills the cages for name and password accordingly and presses the register button. 3. The system on background checks the uniqueness and the righteousness of the filled cages. 3.1 The system creates the account or declines and passes the data to the database.
Post conditions	User has to remember their data, because forgetting password or username options are not available.

3.2 <Login>

Use case ID	loginUC
Actors	a user of the application
Pre conditions	Just the user must register before login.
Main flow of events	1. The use case starts when the user connects to the localhost page on internet. 2. User fills the cages for name and password accordingly and presses the login button. 3. The system on background checks the righteousness of the filled cages.

	4. System accepts user or declines.
Post conditions	User must remember their data, because forgetting password or username options are not available.

3.3 <logout>

Use case ID	logoutUC
Actors	a user of the application
Pre conditions	User must have logged in.
Main flow of events	1. The use case starts when the user presses the logout option on the navigation bar. 2. The system kicks out the connected user.

3.4 <CreateProfile >

Use case ID	createProfileUC
Actors	user of the application
Pre conditions	Login User case.
Main flow of events	1. The use case starts when the user presses the create profile button 2. User fills the cages required. 3. The system on background reads the information provided and creates the profile for the specific user. 4. The system creates the profile by passing the data to the database.
Post conditions	

3.5 <AddOffer>

Use case ID	addOfferUC
Actors	user of the application
Pre conditions	Profile creation.
Main flow of events	1. The use case starts when the user presses the create add offer button of the menu buttons. 2. User fills the cages required. 3. The system on background reads the information provided and creates the book offer for the specific user.

3.6 <DeleteOffer>

Use case ID	deleteOfferUC
Actors	user of the application
Pre conditions	Book offer has been created.
Main flow of events	1. The use case starts when the user presses 'View your offers' button and then the "Delete offer" button near the information of the book offered. 2. The system on background deletes the book from the users' book offers and so from the database.

3.7 <BrowseRequests>

Use case ID	browseRequestsUC
Actors	user of the application, requesters.
Pre conditions	A request has been made for a book offer of the connected user.
Main flow of events	1. The use case starts when the user presses the "View requests on your book offers" button. 2. The systems provide a list of the titles of users' book offers with the option to view for a specific book the requestors. 3. Upon clicking the "View Requests" button systems provides the requestors.

3.8 <AccesssRequestorsInformation>

Use case ID	accessRequestorsInformationUC
Actors	user of the application, requesters.
Pre conditions	User browses the requests of his book offers.
Main flow of events	1. The use case starts when the user presses the “View requests” button for a specific book. 2. The systems provide a list of the contact information of the requestors.

3.9 <NotifyUsers>

Use case ID	notifyUsersUC
Actors	User who accepts and the corresponded users.
Pre conditions	A user must have accepted a book request first
Main flow of events	1. The use case starts when the user presses the “Accept” button for a specific request. 2. The systems work on creating notifications for the users interested in the book accordingly.

3.10 <ViewNotifications>

Use case ID	viewNotificationsUC
Actors	Connected User.
Pre conditions	A user must have accepted a book request first.
Main flow of events	1. The use case starts when the user presses the “Notifications” button for a specific request. 2. The systems provides the corresponding message.

3.11 <Request>

Use case ID	requestUC
Actors	user of the application.
Pre conditions	User has searched a book offer or got a recommendation.
Main flow of events	1. The use case starts when the user presses the “Request book” button. 2. The systems updates the information for the user requested and the user offering the book. 3. The system informs the user if his request was successful.

3.12 <ViewYourRequests>

Use case ID	viewYourRequestsUC
Actors	Connected User.
Pre conditions	A user must have made a book request first.
Main flow of events	1. The use case starts when the user presses the “View your Requests” button for a specific request. 2. The systems provides a list with the requests user has currently made.

3.13 <DeleteRequest>

Use case ID	deleteRequestUC
Actors	Connected User.
Pre conditions	A user must have made a book request first.
Main flow of events	1. The use case starts when the user presses the “View your Requests” button for a specific request. 2. The systems provide a list with the requests user has currently made. 3. After user click on “Delete request” button the system deletes the request from the background.

3.14 <SearchBook>

Use case ID	searchBookUC
Actors	Connected User.
Pre conditions	Another user must have made a book offer in the bookstore application, else no results will be found.
Main flow of events	1. The use case starts when the user presses the “Search Engine” button on the menu. 2. The systems provides a two cages, on for the title and one for the author/s to be filled by the user. 3. The systems also provides 2 search mechanisms. 3.1 When “Exact search” is selected, both title and author/s must match the users input in order the system to return a list of results. 3.2 When “Approximate Search” is selected, the system looks for books containing the title of the users input and the exact authors of it.

3.15 <RecommendBook>

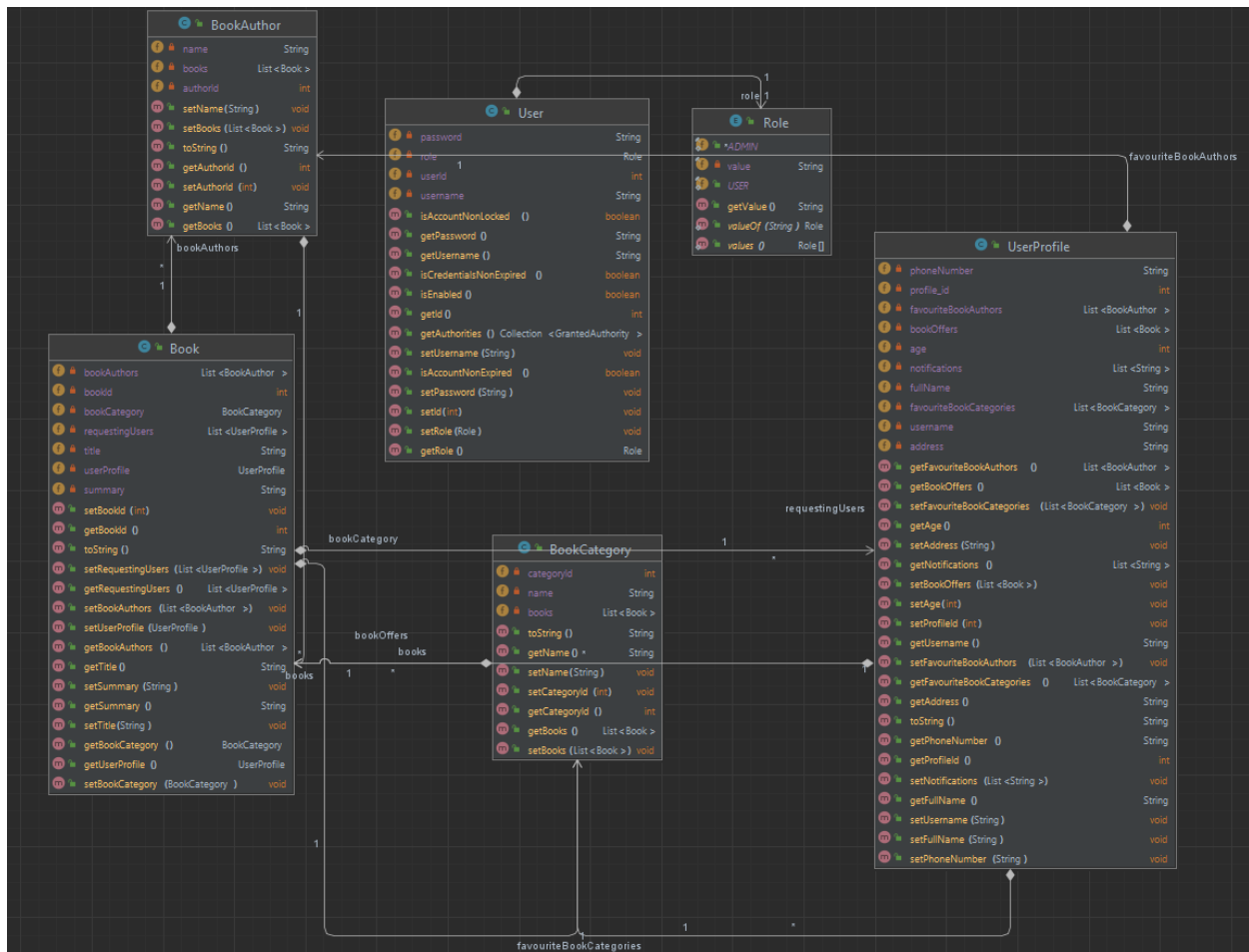
Use case ID	recommendBookUC
Actors	Connected User.
Pre conditions	Another user must have made a book offer in the bookstore application; else no results will be found.
Main flow of events	1. The use case starts when the user presses the “Recommend Engine” button on the menu. 2. The systems provide a three cases for the user to choose what kind of search it will carry out. 3. The systems trigger and responds accordingly. a. When “Recommend by Category” button is pressed, system returns books that match with any of the user’s favorite book categories. b. When “Recommend by Author” button is pressed, the system returns books that match with any of the user’s favorite book authors. c. When “Both” button is pressed, the system search and return books that match any of the credentials.

4 Design

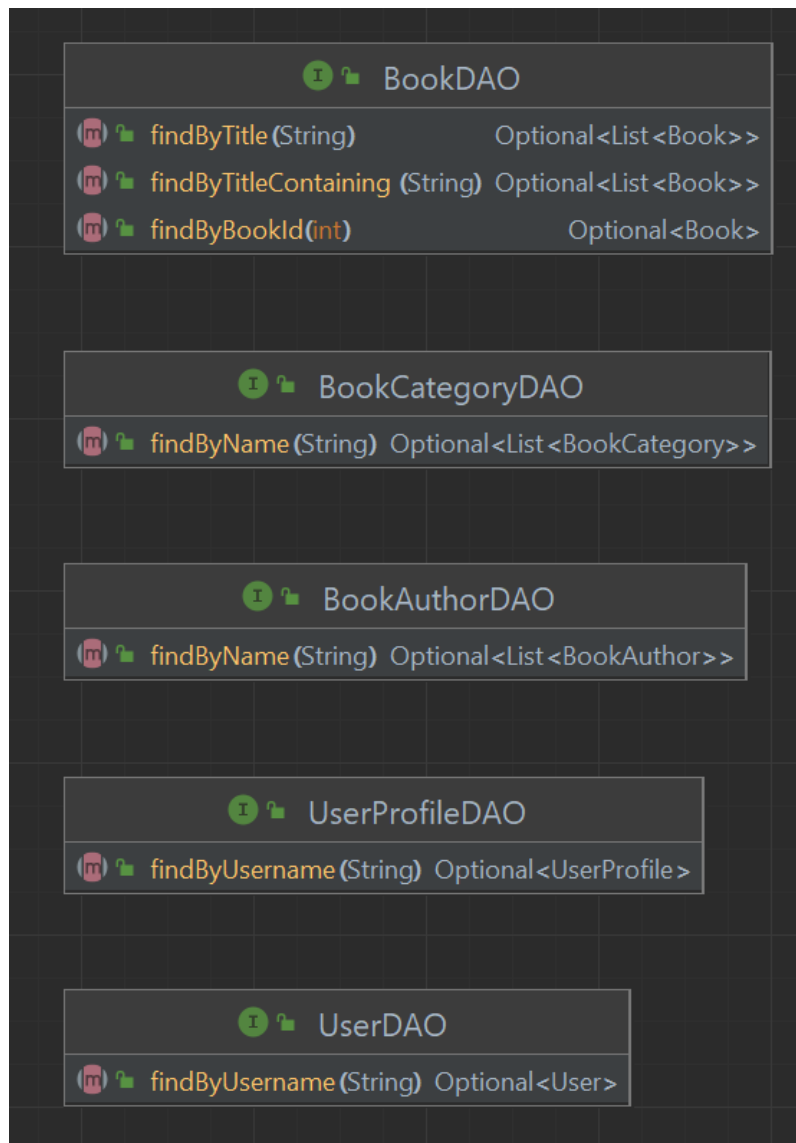
4.1 Architecture

4.2 Design

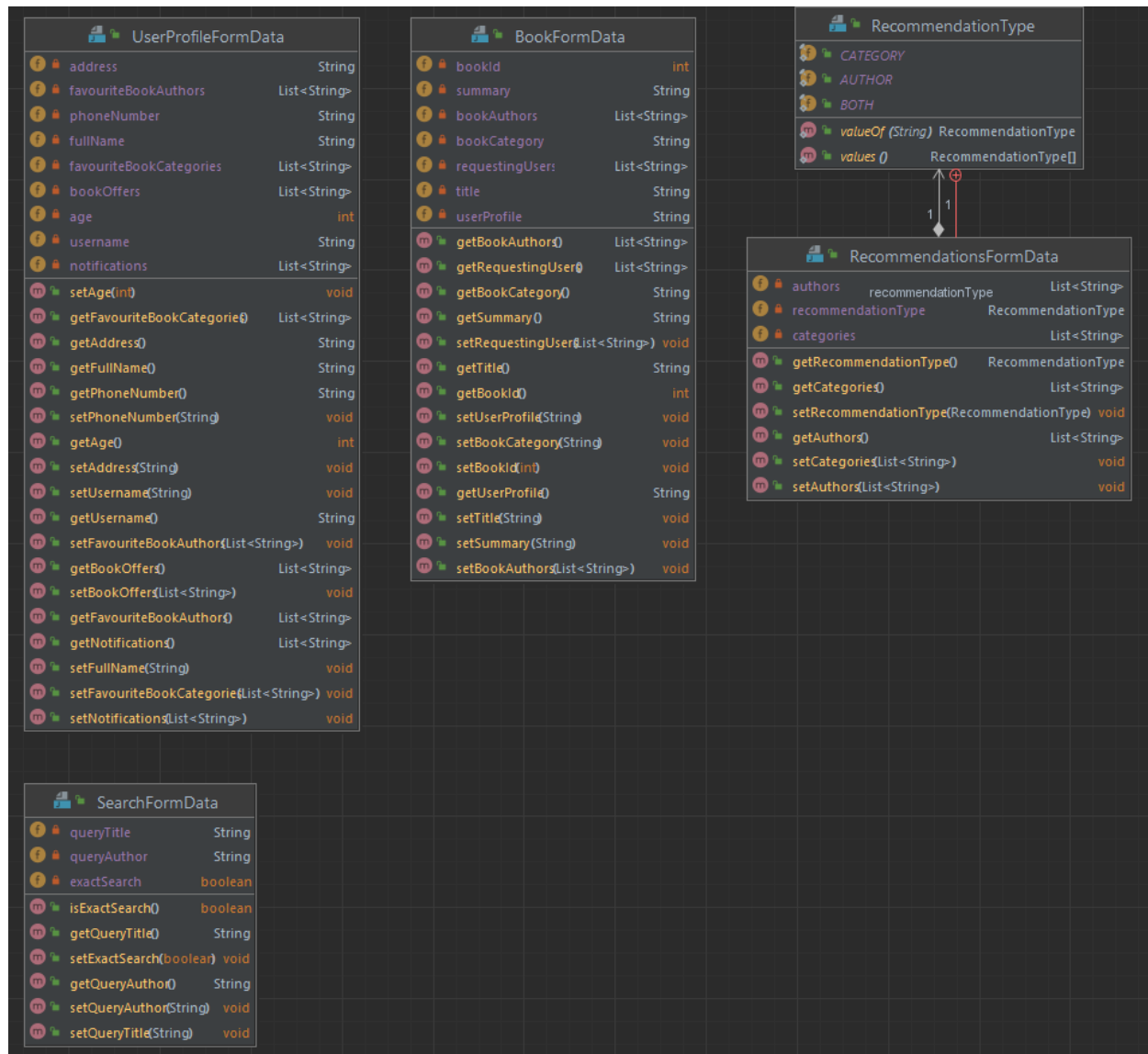
UML of the classes contained in the domain package



UML of the classes contained in the dao package



UML of the classes contained in the formsdata package



[illegible]

UserController		
  userProfileService	UserProfileService	
  userService	UserService	
  showRecommendationsForm(Model)	String	
  deleteBookOffer(int, RedirectAttributes)	String	
  getUserMainMenu()	String	
  getProfileMenu()	String	
  showRequestingUsersForBookOffer(int, Model)	String	
  showUserNotifications(Model)	String	
  showRequesters(Model)	String	
  search(SearchFormData, Model)	String	
  showOfferForm(Model)	String	
  requestBook(int, Model)	String	
  acceptRequest(String, int, RedirectAttributes)	String	
  showUserBookRequests(Model)	String	
  recommendBooks(RecommendationsFormData, Model)	String	
  showSearchForm(Model)	String	
  createProfile(Model)	String	
  deleteBookRequest(String, int, Model)	String	
  listBookOffers(Model)	String	
  saveOffer(BookFormData, Model)	String	
  saveProfile(UserProfileFormData, Model)	String	
  retrieveProfile(Model)	String	

AuthController		
  userService	UserService	
  registerUser(User, Model)	String	
  showHomepage(Model)	String	
  login()	String	
  register(Model)	String	
  logout()	String	